

1. Dati del progetto

- **Nome e Cognome:** Carlo Apeno
- **Classe:** 3B TLC
- **Titolo del gioco:** Tiro al bersaglio
- **Data della consegna:** 3 Giugno 2026

2. Idea del gioco

In cosa consiste il gioco?

L'idea di base è il **Luna Park** e ho deciso di scegliere un classico gioco di abilità Shooting Gallery (il classico tiro al bersaglio dei baracconi del Luna Park), ma rivisitato in chiave ironica e scolastica. Il giocatore si ritrova davanti a un grande palco scorrevole dove, al posto delle solite paparelle di gomma o dei bersagli finti, compaiono i professori della propria scuola, trasformati per l'occasione in "Prof-Mostri"!

Obiettivo del giocatore: come si vince e come si perde?

- **Per vincere:** Il giocatore deve armarsi di riflessi pronti e mouse alla mano. Deve ripulire il palco sconfiggendo uno alla volta i 4 professori intermedi (ciascuno con i propri punti vita e la propria materia scolastica) e superare l'ostacolo finale: lo scontro con il temibile Boss dell'Informatica, il Re dei Bug.
- **Per perdere:** Non è una passeggiata. I professori non stanno fermi a guardare, ma lanciano continui attacchi sotto forma di proiettili/palline distruttive. Se il giocatore non si sposta lateralmente per schivarli o finisce i colpi nel caricatore nel momento sbagliato, la sua barra della vita scende a zero, attivando il Game Over e la frase: "Niente vacanze estive per te, ci vediamo al corso di recupero!"

Cosa lo rende originale?

Rispetto ai classici giochi di tiro al bersaglio, questo progetto ha alcune caratteristiche che lo rendono diverso:

- **Ambientazione scolastica personalizzata:** i nemici sono professori realmente conosciuti dagli studenti e ogni personaggio ha frasi e riferimenti legati alla propria materia. Questo rende il gioco più divertente e vicino all'esperienza scolastica.

- **Movimento nel livello:** il giocatore non rimane fermo in una schermata, ma può spostarsi a destra e sinistra per inseguire i professori. Questo rende il gameplay più dinamico.
- **Bonus e sistema combo:** durante la partita possono comparire bonus che curano il giocatore o raddoppiano i punti. Inoltre, colpire più volte consecutivamente i nemici aumenta la combo e permette di ottenere punteggi più alti.
- **Boss finale:** dopo aver sconfitto tutti i professori compare un boss finale più forte e veloce, che rappresenta la sfida conclusiva del gioco.

Pubblico di riferimento

Il gioco è pensato principalmente per gli studenti e i compagni di classe, che possono riconoscere i professori e le battute presenti nel gioco.

Tuttavia, grazie alle sue meccaniche semplici e immediate, può essere apprezzato anche da chiunque voglia giocare a un piccolo sparatutto arcade, cercando di ottenere il punteggio più alto possibile.

3. Analisi e progettazione

3.1 Regole del gioco

Il gioco è uno sparatutto in **prima persona** in cui il giocatore deve eliminare una serie di professori nemici prima di essere sconfitto.

Il giocatore inizia la partita con **100 punti Vita**. I professori attaccano lanciando delle palline rosse che si avvicinano progressivamente allo schermo. Se una pallina raggiunge il giocatore, vengono sottratti **15 punti Vita** e viene mostrato un effetto visivo rosso per segnalare il danno subito. Quando i punti Vita scendono a 0, la partita termina e viene mostrata la schermata di Game Over.

I nemici compaiono uno alla volta in un ordine prestabilito:

1. **Trioschi**
2. **Cascone**
3. **Fabretto**
4. **Cavallo**

Ogni professore possiede **5 punti vita**, rappresentati da cinque indicatori verdi sopra il personaggio. Ogni colpo andato a segno riduce la sua vita di un punto. Dopo l'eliminazione di un professore compare il successivo, che si muove più velocemente rispetto al precedente.

Dopo aver sconfitto i quattro professori normali compare il **Boss finale, Mirto Musci**, che possiede 10 punti vita, si muove molto più velocemente e dispone di una barra della salute dedicata visibile nell'interfaccia del gioco.

La partita viene vinta quando il Boss finale viene sconfitto completamente. In questo caso viene mostrata una schermata di **vittoria**. inoltre il giocatore dispone di una pistola con 6 munizioni. Quando il caricatore è vuoto è necessario premere il **tasto R** per ricaricare. Durante la ricarica non è possibile sparare.

È presente anche un sistema di combo: ogni colpo consecutivo andato a segno aumenta il moltiplicatore della combo e permette di ottenere più punti. Se il giocatore manca un bersaglio o subisce un colpo, la combo viene azzerata.

Durante la partita possono comparire casualmente dei **power-up**:

- Libro di recupero: ripristina 20 punti Vita.
- Gemini: attiva per 5 secondi un moltiplicatore che raddoppia tutti i punti ottenuti.

Tra un livello e l'altro viene mostrata una schermata di transizione con il nome del professore, la materia insegnata e una breve frase caratteristica.

È presente un **sistema di punteggio**:

- 50 punti per ogni colpo inflitto a un professore normale;
- 200 punti bonus per l'eliminazione di un professore normale;
- 100 punti per ogni colpo inflitto al Boss finale.

I bonus, le combo e il moltiplicatore dei punti permettono di ottenere punteggi più elevati, aumentando la competitività del gioco e incentivando la precisione del giocatore.

3.2 Struttura del programma

Il programma è stato sviluppato utilizzando il linguaggio **Python** e la libreria **Pygame**, che permette di gestire grafica, input da tastiera e mouse, immagini, animazioni e collisioni. Inoltre vengono utilizzate altre librerie:

- **random**: per generare posizioni, movimenti casuali e comparsa dei bonus.
- **math**: per creare movimenti più realistici e animazioni (ad esempio l'oscillazione dei bonus).
- **os**: per caricare correttamente le immagini presenti nella cartella del progetto.
- **sys**: per chiudere il programma in modo corretto.

Struttura del programma

- **Menu iniziale:** il giocatore inserisce il proprio nome, può visualizzare la guida dei bonus e avviare la partita.
- **Gestione del giocatore:** il giocatore ha 100 punti vita, una pistola con 6 munizioni, può ricaricare con il tasto **R** e muoversi con **A/D** o le frecce direzionali.
- **Gestione dei nemici:** i professori compaiono uno alla volta, si muovono sullo schermo, hanno 5 punti vita e attaccano lanciando palline. Dopo di loro compare il Boss finale con 10 punti vita e una barra della salute dedicata.
- **Gestione dei danni:** quando una pallina colpisce il giocatore perde 15 punti vita e viene mostrato un effetto visivo di danno.
- **Gestione del punteggio:** il giocatore guadagna punti colpendo e sconfiggendo i nemici. È presente anche un sistema di combo che aumenta i punti ottenuti.
- **Gestione dei bonus:** durante la partita possono comparire il **Libro di recupero** (ripristina vita) e **Gemini** (raddoppia i punti per alcuni secondi).
- **Schermate finali:** se la vita del giocatore arriva a zero viene mostrato il **Game Over**; se il Boss viene sconfitto viene mostrata la schermata di **Vittoria** con il punteggio finale.

Pseudocodice del programma

AVVIO DEL PROGRAMMA

- Inizializzazione di Pygame e delle variabili principali.
- Caricamento delle immagini, dei personaggi e delle risorse grafiche.
- Visualizzazione del menu iniziale.

MENU INIZIALE

- Inserimento del nome del giocatore.
- Possibilità di visualizzare la guida dei bonus.
- Avvio della partita tramite il tasto INVIO.

INIZIALIZZAZIONE DELLA PARTITA

- Impostazione della vita del giocatore a 100.
- Azzeramento del punteggio e delle combo.
- Caricamento del primo professore.
- Visualizzazione della schermata di transizione.

CICLO PRINCIPALE DI GIOCO

- Lettura degli input da tastiera e mouse.
- Gestione del movimento della telecamera.

- Gestione degli spari del giocatore.
- Gestione della ricarica delle munizioni.
- Aggiornamento della posizione dei nemici.
- Generazione degli attacchi dei nemici.
- Controllo delle collisioni tra colpi e bersagli.
- Aggiornamento della vita del giocatore e dei nemici.
- Aggiornamento del punteggio e delle combo.
- Gestione dei bonus e dei power-up.
- Aggiornamento degli effetti grafici.
- Disegno di tutti gli elementi sullo schermo.

GESTIONE DEI LIVELLI

- Se il professore viene sconfitto:
 - assegnazione del bonus punti;
 - caricamento del professore successivo;
 - visualizzazione della schermata di transizione.
- Se tutti i professori vengono sconfitti:
 - avvio dello scontro con il Boss finale.

GESTIONE DEL BOSS

- Movimento più veloce del Boss.
- Attacchi più frequenti.
- Aggiornamento della barra della salute del Boss.
- Controllo della sua eliminazione.

CONTROLLO DELLE CONDIZIONI DI FINE PARTITA

- Se la vita del giocatore raggiunge 0:
 - visualizzazione della schermata di Game Over.
- Se la vita del Boss raggiunge 0:
 - visualizzazione della schermata di Vittoria.

CHIUSURA DEL PROGRAMMA

- Chiusura di Pygame.
- Terminazione del gioco.

3.3 Strutture dati importanti

Per organizzare le informazioni del gioco ho utilizzato principalmente **liste**, **dizionari** e **variabili**.

- **dati_professori**: è una lista che contiene i quattro professori nemici. Per ogni professore vengono salvati il nome, l'immagine, la posizione sul palco, la direzione di movimento, la velocità e i punti vita.
- **dati_boss**: è un dizionario che contiene le informazioni del boss finale Mirto Musci, come nome, immagine e punti vita.
- **palline_mostri**: è una lista che contiene tutte le palline lanciate dai nemici. Per ogni pallina vengono memorizzate la posizione iniziale, la destinazione e lo stato dell'animazione.

- **lista_powerup**: è una lista che contiene tutti i bonus presenti sullo schermo. Per ogni bonus vengono salvati il tipo, la posizione e la velocità di movimento.
- **lista_testi_fluttuanti**: è una lista utilizzata per mostrare sullo schermo i punti guadagnati dopo aver colpito un nemico. Ogni elemento contiene il testo da visualizzare, la posizione e il livello di trasparenza.
- **stelle_sfondo**, **luci_luna_park** e **particelle_sfondo**: sono liste utilizzate per creare gli effetti grafici dello scenario e rendere l'ambientazione più dinamica.
- Alcune variabili come **vita_giocatore**, **punteggio**, **prof_eliminati**, **combo_attuale**, **munizioni_attuali** e **STATO_GIOCO** vengono utilizzate per tenere traccia dell'andamento della partita e determinare cosa deve essere mostrato sullo schermo.

Per questo progetto non ho utilizzato classi, ma ho scelto di gestire i dati tramite liste e dizionari, che erano sufficienti per la complessità del gioco. Questa soluzione ha reso il codice più semplice da comprendere e da modificare durante lo sviluppo.

4. Sviluppo e uso dell'AI

4.1 Come ho lavorato

Ho sviluppato il progetto individualmente. Per prima cosa ho pensato all'idea del gioco e al tema generale, scegliendo di realizzare uno sparatutto in prima persona ambientato in una versione ironica di tiro al bersaglio.

Successivamente ho progettato le regole principali del gioco, decidendo come gestire la vita del giocatore, il sistema di punteggio e lo scontro con i prof e il boss finale.

Una volta definita la struttura, ho iniziato a sviluppare il codice utilizzando Pygame. Ho realizzato prima il menu iniziale e il sistema di movimento della telecamera, poi ho aggiunto i professori, il sistema di sparo, le collisioni, il punteggio e infine gli effetti grafici e le schermate di vittoria e sconfitta.

- **Le parti che mi hanno richiesto più tempo sono** state la gestione delle palline lanciate dai nemici e mettere le decorazioni dietro in movimento
- **La difficoltà principale:** È stata coordinare tutti gli elementi contemporaneamente, facendo in modo che movimenti, punteggio, vita, nemici ed effetti grafici funzionassero correttamente insieme, senza creare conflitti o errori bloccanti durante la partita.

4.2 Come ho usato l'AI

Per lo sviluppo di questo progetto ho utilizzato l'intelligenza artificiale (Gemini), principalmente per il codice che poi mi avrebbe portato alla nascita del gioco.

Per cosa ho usato l'AI:

Ho utilizzato l'AI per generare esempi di codice base, soprattutto per la gestione della telecamera e dei movimenti dei proiettili. Mi è stata utile anche per correggere errori legati alle coordinate e ai tipi di dato, aiutandomi a individuare più velocemente i bug. Inoltre l'ho usata per ottimizzare e semplificare il codice, in particolare per organizzare i professori e il boss tramite liste e dizionari, evitando di scrivere codice ripetitivo.

Alcuni prompt principali che ho utilizzato:

- "Aiutami a creare una funzione di sicurezza in Python per caricare le immagini (.png) senza far crashare il programma se i file non si trovano nella cartella."
- "Mostrami un esempio in Pygame di come muovere la telecamera a destra e a sinistra allungando il palco oltre la finestra visibile."
- "Ho un errore di collisione quando sposto la visuale, come faccio a calcolare il click del mouse considerando lo spostamento di camera_x?"

Come ho modificato e fatto mio il codice dell'AI:

Non mi sono limitato a utilizzare il codice così com'era, ma l'ho personalizzato e modificato per adattarlo al mio progetto, cambiando personaggi, regole di gioco, punteggi, punti vita e altre funzionalità.

Ho semplificato diverse parti che erano troppo complesse o non necessarie, eliminando elementi che avrebbero appesantito il programma e rendendolo più chiaro e gestibile.

Inoltre ho cercato di capire il funzionamento del codice prima di inserirlo nel progetto, analizzando le varie parti e testandole più volte. Questo mi ha permesso di avere un controllo maggiore sul programma e di imparare meglio come funziona la logica di un videogioco in Python.

5. Test e risultati

5.1 Casi di test

Ho verificato il funzionamento del gioco testando le regole principali sia in situazioni normali che di errore. Ecco il riassunto dei test:

ID	Cosa ho fatto	Cosa mi aspettavo	Cosa è successo	Stato
1	Avvio il gioco senza il file <code>pistola.png</code> .	Il gioco non deve andare in crash.	Disegna un poligono grigio di riserva.	Risolto
2	Subisco il colpo di una pallina.	Perdere 15% di vita e flash rosso.	Vita scalata e flash visivo funzionante.	Risolto
3	Clicco su un professore attivo.	Togliere 1 punto vita e ricevere 50 punti.	Ha funzionato; al 5° click il prof scompare.	Risolto
4	Sconfiggo il 4° prof (Cavallo).	Deve apparire il Boss <i>Mirto Musci</i> .	Lo stato è passato a "BOSS" con l'HUD rosso.	Risolto
5	Clicco sulla X della finestra.	Il gioco si deve chiudere subito.	Chiusura corretta senza errori.	Risolto

5.2 Bug e problemi

- **Spam del Click (Rateo di fuoco):** Non essendoci un tempo di recupero (*cooldown*) tra un colpo e l'altro, un giocatore molto veloce può svuotare il caricatore da 6 colpi in pochi millisecondi, togliendo molta sfida al gioco.
- **Parti incomplete (Tempo insufficiente):** * **Assenza di Audio:** Il gioco è totalmente silenzioso; avremmo voluto inserire i suoni degli spari, della ricarica e delle musiche di sottofondo.
- **Mancanza di tempo:** Volevo inserire un sistema di salvataggio dei record su un file di testo esterno `.txt` per fare una classifica tra i compagni della 3B TLC, ma l'ho dovuto tagliare per evitare rallentamenti nel gioco e consegnare il progetto in tempo.

6. Conclusioni e riflessione personale

In questo progetto ho creato un videogioco in Python utilizzando la libreria Pygame. È stata un'esperienza molto bella e utile perché mi ha permesso di capire come funziona la programmazione quando si lavora su un progetto completo e più complesso rispetto ai semplici esercizi svolti in classe. Durante lo sviluppo ho dovuto mettere insieme molte parti diverse come la grafica, il movimento dei personaggi e il sistema di punteggio e tutto ciò mi ha fatto capire meglio quanto sia importante organizzare bene il lavoro fin dall'inizio e procedere in modo ordinato perché sennò si possono creare problemi poi nel corso del lavoro.

Che cosa ho imparato di nuovo?

Ho imparato a usare meglio Python e la libreria Pygame, soprattutto per creare grafica, gestire input da tastiera e mouse e aggiornare lo schermo in tempo reale. Inoltre ho capito come organizzare meglio un programma complesso usando liste e dizionari per gestire in modo ordinato elementi come nemici, punteggio e vita del giocatore. Infine un'altra cosa importante che ho imparato è che quando si usa l'intelligenza artificiale è fondamentale saper fare richieste chiare e precise, perché se i prompt non sono ben scritti il risultato può essere incorretto e il lavoro non verrà mai.

Qual è stata la cosa più difficile?

La parte più difficile è stata coordinare tutte le componenti del gioco, perché anche un piccolo errore poteva compromettere il funzionamento dell'intero programma. Inoltre è stato particolarmente impegnativo realizzare e sincronizzare i movimenti delle decorazioni e degli effetti presenti sullo sfondo del palco.

Sono riuscito a superare una difficoltà? Come?

Sì, dopo diversi tentativi e prove sono riuscito a risolvere il problema, ho ricontrollato più volte il codice e ho corretto il modo in cui venivano calcolate le posizioni degli oggetti rispetto alla telecamera. Finché facendo piccoli test ogni volta sono riuscito a capire dove sbagliavo e alla fine il sistema ha iniziato a funzionare in modo corretto e coordinato.

Se rifacessi il progetto cosa faresti in modo diverso?

Se dovessi rifare il progetto dedicherei più tempo alla fase iniziale di progettazione. Cercherei di organizzare meglio le idee prima di scrivere il codice e procederei passo per passo testando ogni parte subito dopo averla creata. In questo modo eviterei di accumulare errori e di dover correggere tutto alla fine. Poi cercherei dei siti che mi aiutino a comprendere meglio i comandi nuovi che AI mi dà così da capire ancora in modo più veloce e approfondito i comandi nuovi e capire come utilizzarli.

Pensi di aver usato l'AI in modo intelligente o ti sei appoggiato troppo?

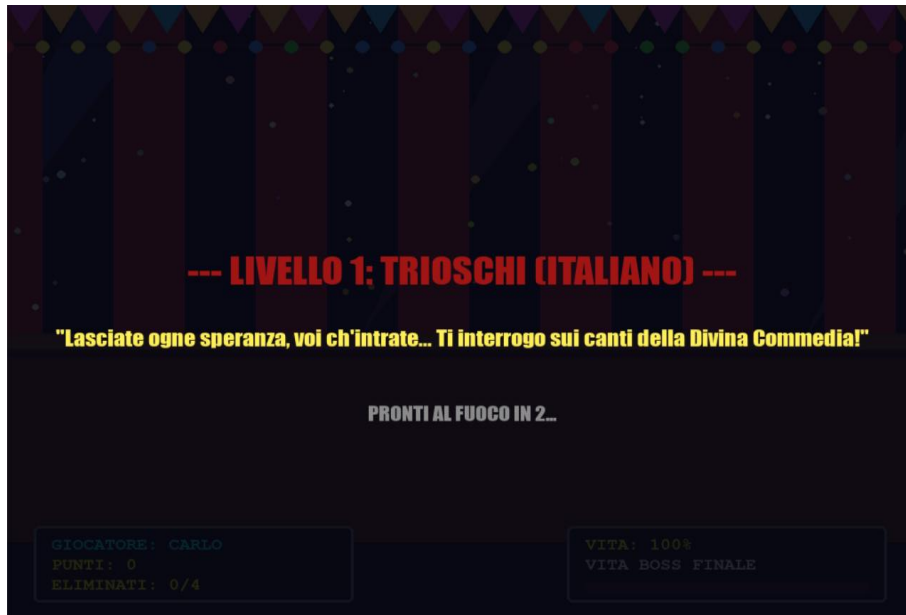
Per la costruzione del progetto ammetto che mi sono appoggiato abbastanza all'Intelligenza Artificiale soprattutto perché alcune parti non sapevo proprio come farle visto che a scuola abbiamo studiato solo le "basi" di Python. Nonostante questo però penso di averla usata in modo intelligente. Infatti non ho detto a Gemini "Fammi il codice" e basta ma ho fatto diversi test ho cercato di capire gli errori e ho usato l'AI soprattutto per correggere e comprendere meglio il funzionamento del programma.

P.S. E stata un'esperienza nuova e molto bella anche se a causa del poco tempo ci siamo dovuti limitare molto. Per il futuro spero che il prossimo anno riesca a creare un gioco semplice completamente mio senza l'uso di Gemini o di altre intelligenze artificiali.

7. Allegati



Commentato [1]: Figura 1: Schermata iniziale con la casella di testo per l'inserimento del nome del giocatore, con guida bonus.



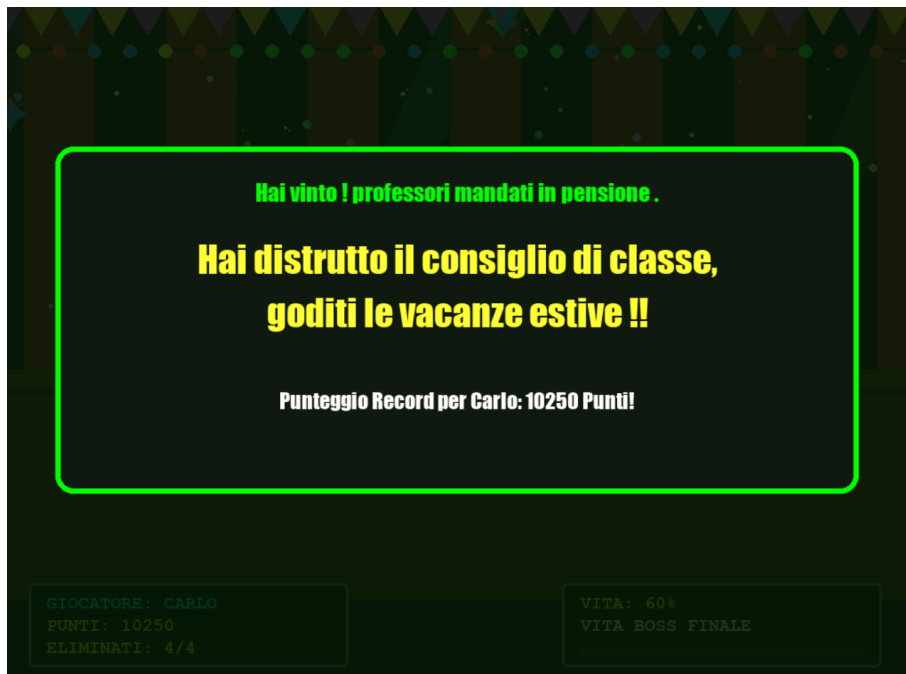
Commentato [2]: Figura 2: Schermata di transizione (Prof. Trioschi).



Commentato [3]: Figura 3: Schermata di gioco. Mostra la partita in corso contro la Prof. Cascone. Al centro ci sono il mirino e un proiettile rosso in arrivo; in basso si vede l'interfaccia con i punti (750), la vita rimasta (70%) e i colpi nel caricatore (1 su 6).



Commentato [4]: Figura 4: Scontro finale con il Boss. Mostra la battaglia finale contro il boss Mirto Musci, mentre in basso a destra si attiva la barra della salute rossa dedicata interamente al Boss.



Commentato [5]: Figura 5: Schermata di vittoria. Mostra il messaggio di fine partita quando si vince il gioco. Appare un riquadro verde con i complimenti ironici ("Hai distrutto il consiglio di classe, goditi le vacanze estive !!") e viene mostrato il punteggio record finale ottenuto dal giocatore (10250 punti).



Commentato [6]: Figura 6: Schermata di Game Over. Mostra la fine della partita quando i punti vita del giocatore scendono a zero. Appare una schermata scura con la scritta rossa "GAME OVER" e una frase ironica che annuncia la bocciatura o il corso di recupero estivo.

Codice:

[Codice relazione informatica](#)